

https://docs.google.com/presentation/d/1hY1lcNvcWUMfN3an7kn0-hnaFx-dfLqyslW2nz7BEbQ/edit#slide=id.g51da6455d5_0_0

Ethers: Bringing Ethereum to the Web

Course Leads:

Sara Magaziotis-Ginori

Olivia Li

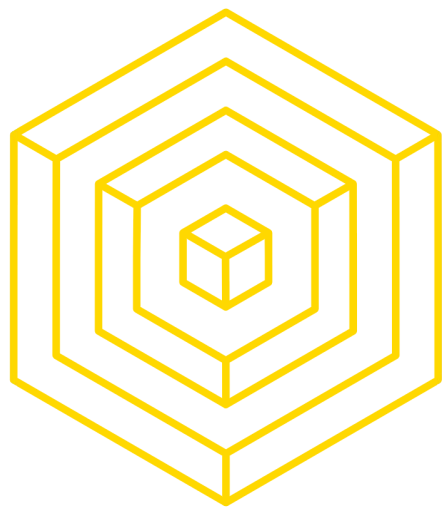
Isaac Oh

Aman Shah

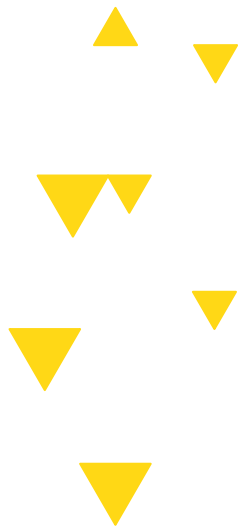
Elson Liu

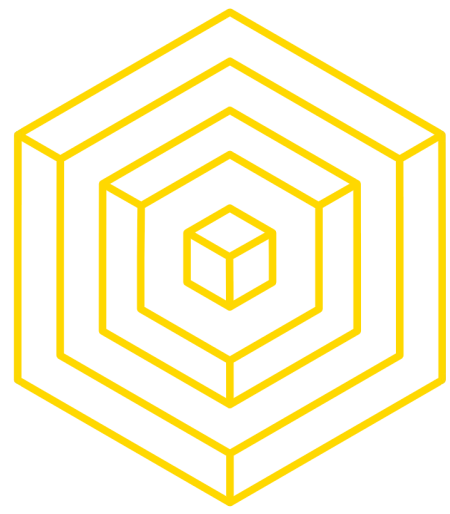
Fujia Wang





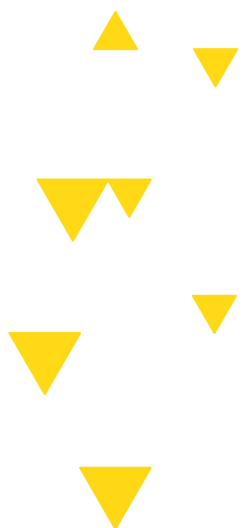
O Announcements





Announcements

- Homework #3 (Implementing NFT Marketplace)
 - Due today at 11:59PM
- Homework #4 (Etherjs)
 - Already released on the Github
 - Due next Wednesday at 11:59PM
- All submission links are posted on the Discord and Github
- Email dev-decal@blockchain.berkeley.edu for extensions





LECTURE OUTLINE

- 1  ETHEREUM ARCHITECTURE
- 2  ETHERS.JS OVERVIEW
- 3  ETHERS.JS FUNCTIONS
- 4  DEMO

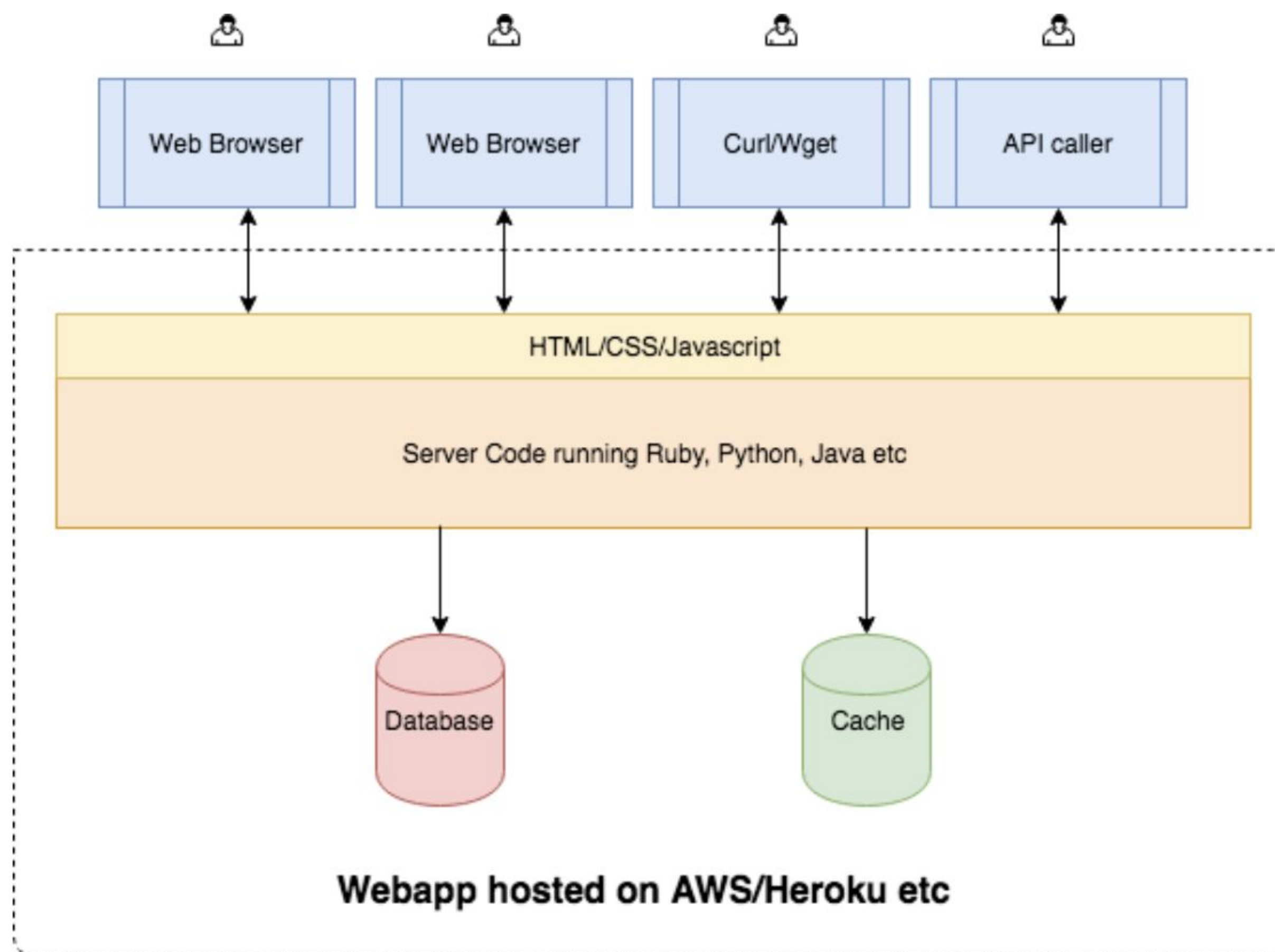


1 ETHEREUM ARCHITECTURE



STANDARD WEB

LOTS OF BOXES



What the user interacts with

What turns user interactions into program interactions and displays the results

The actual programming logic

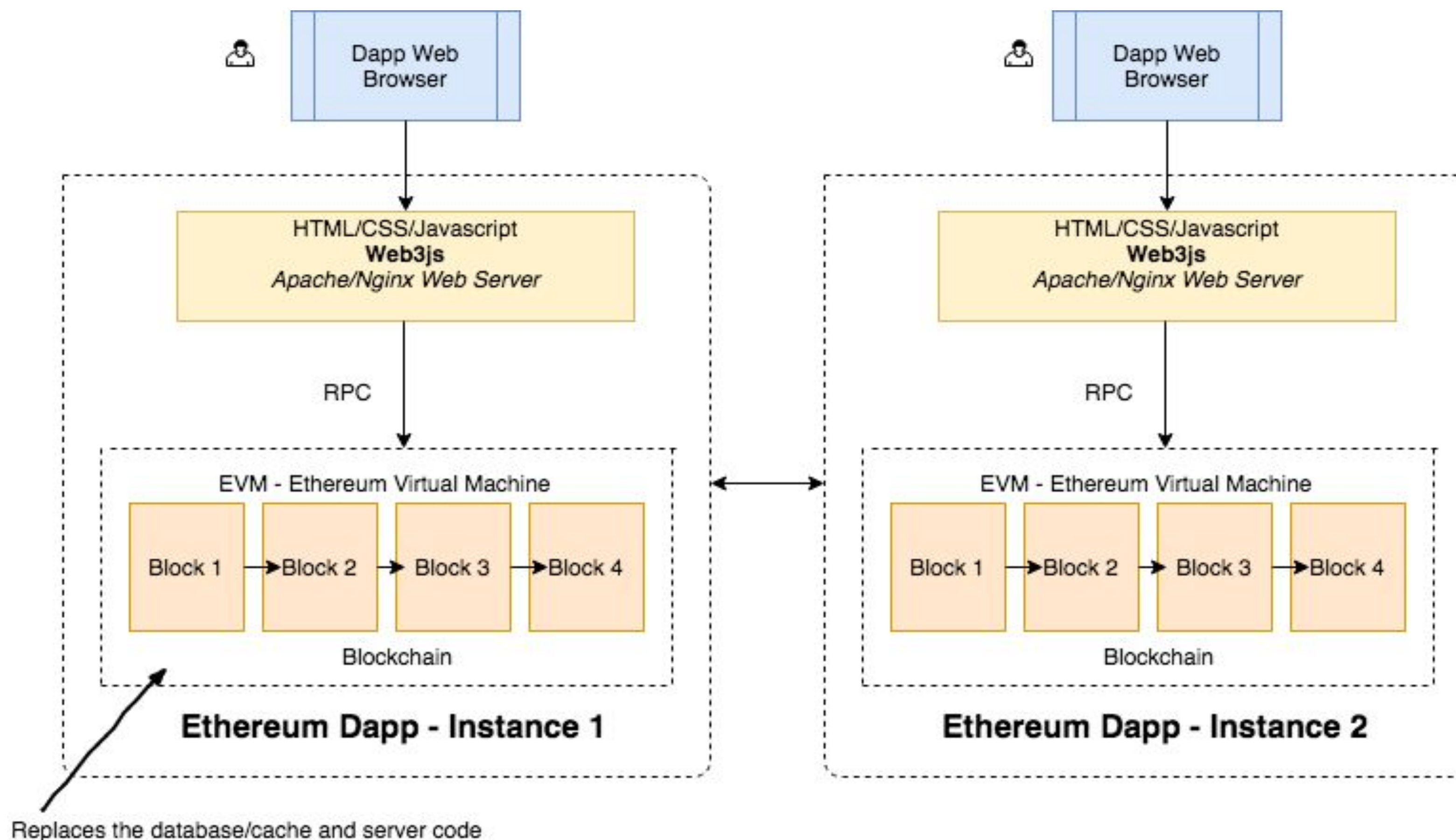
Stores the data

Cloud provider manages infra



WHAT DOES ETHEREUM LOOK LIKE?

LOTS OF BOXES



What the user interacts with

What turns user interactions into program interactions and displays the results

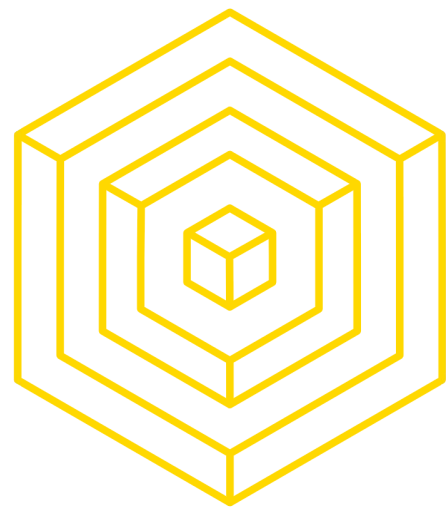
The actual programming logic (smart contracts)



HOW DO WE DO THIS?

ETHERS.JS

- Ethers.js is the entrance to the Ethereum blockchain from the client side of a dApp.
- Communicates to nodes on network using RPC (Remote Procedure Calls)

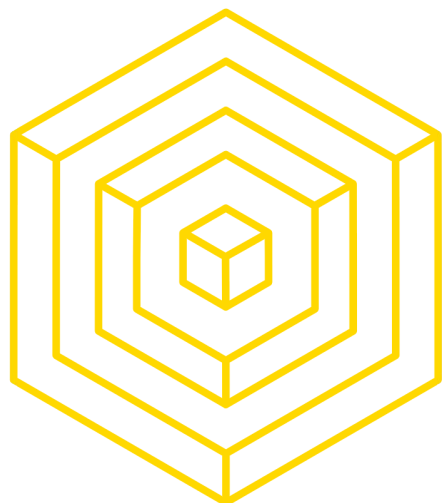


HIGH-LEVEL EXAMPLE

WHAT GOES ON UNDER THE HOOD

Let's say you want to use a dApp that allows you to vote for candidate A or B. Here's an idea of what happens when you click "A" and send the transaction:

1. Frontend receives input
2. Relevant inputs and functions (let's say `Vote("A")`) are passed into ethers.js contract instance, matched against ABI
3. Transactions and events are fired
4. Ethers instance queries blockchain for updated data (`votes["A"] = 1`)
5. New data displayed on frontend



2 ethers.js OVERVIEW



WHAT IS ETHERS.JS?

POKE POKE

ethers

a collection of libraries that allows a user to **interact** with the Ethereum platform and smart contracts



DIFFERENT IMPLEMENTATIONS

ETHERS DOT...

Python **ethers.py**

Rust **ethers.rs**

Easy to make, most used languages now have some kind of implementation in progress, but we will mainly focus on:

JavaScript **ethers.js**



WHAT CAN ETHERS.JS DO?

EVERYTHING

- Interfaces with Ethereum nodes from the network using **JSON-RPC** calls
- We use ethers to:
 - Interact with contract functions
 - Deploy contracts
 - Send data to contracts and initiate transactions
 - Query the blockchain for data
 - Includes logged events by any contract along with block data



ETHERS CLASSES

Basics

- Ethers defines a few classes with which we can separate concerns:
 - **Provider**
 - A class which provides an abstraction for a connection the the Ethereum Network
 - **Signer**
 - A class which in some way directly or indirectly has access to a private key to sign transactions with
 - **Contract**
 - An abstraction which represents a connection to a specific contract on Ethereum



WHAT IS RPC?

HOW DO WE “CONNECT TO ETHEREUM”?

RPC

Remote Procedure Call (RPC) is a protocol that one program can use to **request a service** from a program located in another computer in a network without having to understand network details.

[SOURCE](#)



JSON-RPC API

CONTEXT BEHIND ETHEREUM'S API

- JSON is a lightweight data-interchange format
- RPC is used to make remote function calls
- JSON-RPC is a stateless, lightweight remote procedure call (RPC) protocol.
 - Defines several data structures and the rules around their processing.
 - It is transport agnostic in that the concepts can be used within the same process, over sockets, over HTTP, or in many various message passing environments

Side note: Serialization is the process of translating data structures or object state into a format that can be stored (for example, in a file or memory buffer) or transmitted (for example, across a network connection link) and reconstructed later (possibly in a different computer environment).



ETHEREUM ABI

EXPOSING CONTRACT METHODS

ABI = Application Binary Interface

- An ABI is how you **call** functions in a contract and **get data back**
 - It determines how functions are called and in which binary format information should be passed from one program component to the next
- Why is it necessary?
 - You need a way to specify **which function** in the contract to invoke as well as guarantee to **what type of data is returned**
- Not part of the core Ethereum protocol, you can define your own ABI - but easier to comply with the standard format(s)



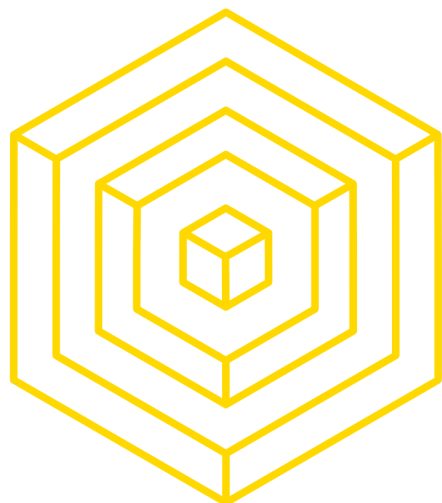
ETHEREUM ABI

ANOTHER ANALOGY

API = Application **Programming** Interface

ABI = Application **Binary** Interface

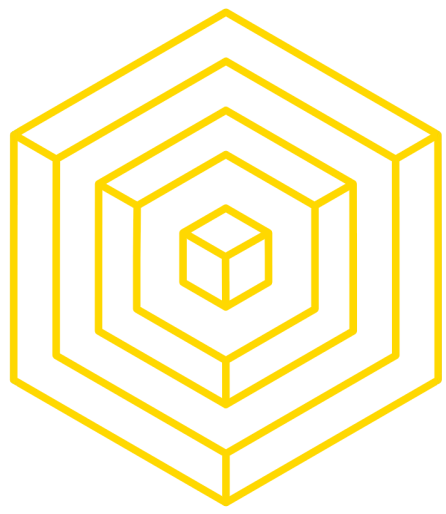
- So therefore an ABI is an API at a lower level?
- Contract code is stored as bytecode in binary form on the blockchain under a specific address
 - You can access the binary data in the contract through the ABI
 - The ABI defines the structures and methods that you will use to interact with binary contract (just like an API)



ABI EXAMPLES

```
contract Test {  
  function Test(){ b = 0x12345678901234567890123456789012; }  
  event Event(uint indexed a, bytes32 b);  
  event Event2(uint indexed a, bytes32 b);  
  function foo(uint a) { Event(a, b); }  
  bytes32 b;  
}
```

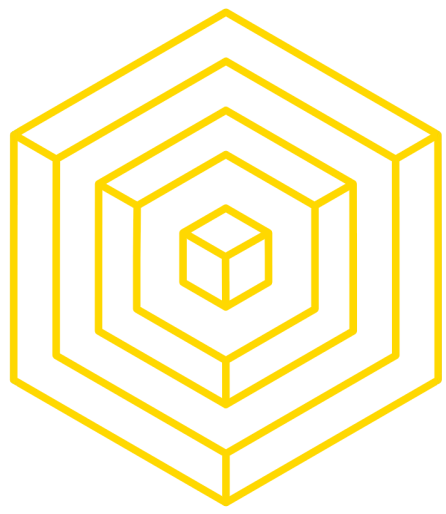
```
[{  
  "type": "event",  
  "inputs": [{"name": "a", "type": "uint256", "indexed": true}, {"name": "b", "type": "bytes32", "indexed": false}],  
  "name": "Event"  
}, {  
  "type": "event",  
  "inputs": [{"name": "a", "type": "uint256", "indexed": true}, {"name": "b", "type": "bytes32", "indexed": false}],  
  "name": "Event2"  
}, {  
  "type": "function",  
  "inputs": [{"name": "a", "type": "uint256"}],  
  "name": "foo",  
  "outputs": []  
}]
```

ABI EXAMPLES

Human-Readable ABI

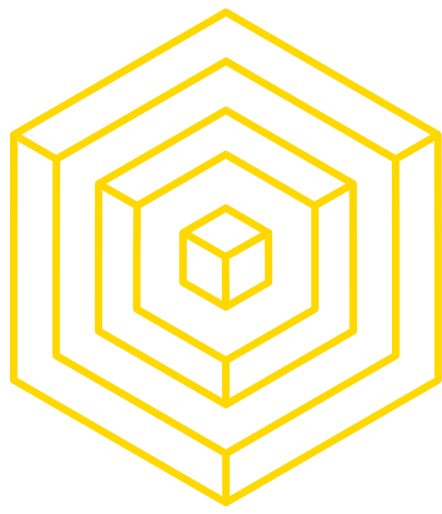
```
const humanReadableAbi = [  
  
  // Simple types  
  "constructor(string symbol, string name)",  
  "function transferFrom(address from, address to, uint value)",  
  "function balanceOf(address owner) view returns (uint balance)",  
  "event Transfer(address indexed from, address indexed to, address value)",  
  "error InsufficientBalance(account owner, uint balance)",  
  
  // Some examples with the struct type, we use the tuple keyword:  
  // (note: the tuple keyword is optional, simply using additional  
  //      parentheses accomplishes the same thing)  
  // struct Person {  
  //   string name;  
  //   uint16 age;  
  // }  
  "function addPerson(tuple(string name, uint16 age) person)",  
  "function addPeople(tuple(string name, uint16 age)[] person)",  
  "function getPerson(uint id) view returns (tuple(string name, uint16 age) person)",  
  
  "event PersonAdded(uint indexed id, tuple(string name, uint16 age) person)",  
];
```

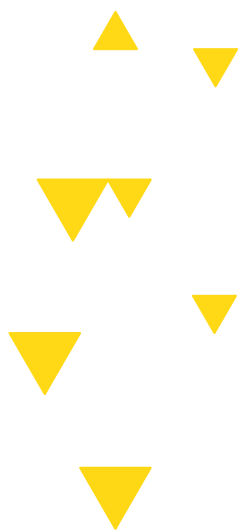
ABI EXAMPLES

Solidity JSON ABI

```
const jsonAbi = `[
  {
    "type": "constructor",
    "payable": false,
    "inputs": [
      { "type": "string", "name": "symbol" },
      { "type": "string", "name": "name" }
    ]
  },
  {
    "type": "function",
    "name": "transferFrom",
    "constant": false,
    "payable": false,
    "inputs": [
      { "type": "address", "name": "from" },
      { "type": "address", "name": "to" },
      { "type": "uint256", "name": "value" }
    ],
    "outputs": [ ]
  },
  {
    "type": "function",
    "name": "balanceOf",
    "constant": true,
    "stateMutability": "view",
    "payable": false, "inputs": [
      { "type": "address", "name": "owner" }
    ],
    "outputs": [
      { "type": "uint256" }
    ]
  },
]
```



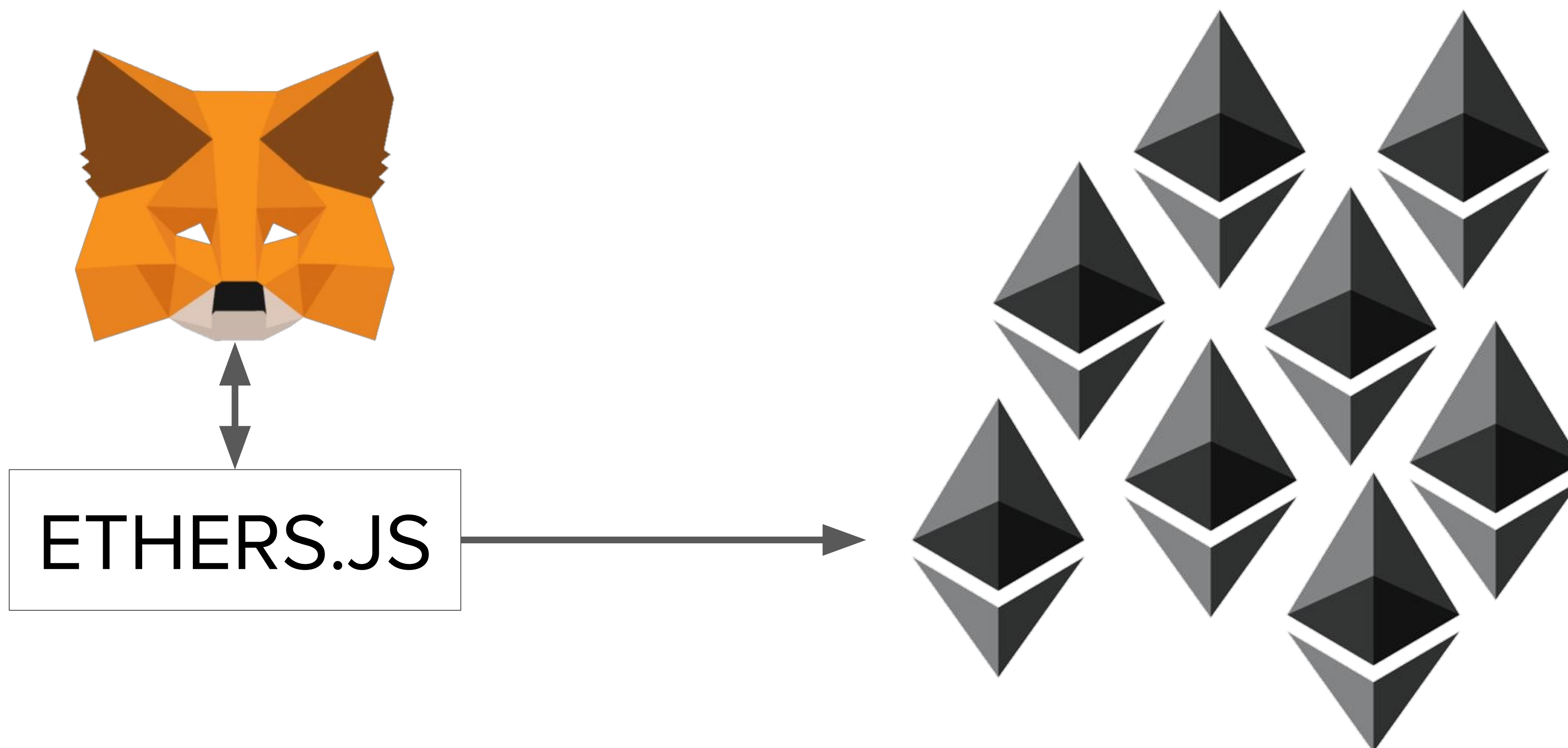
3 ETHERS.JS USAGE

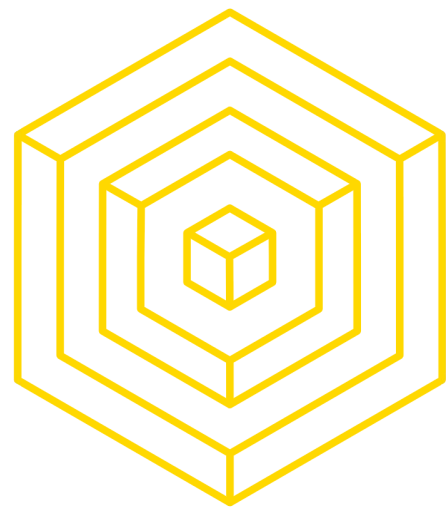




WEB3.JS: Connecting the Blockchain

NOW PUTTING IT ALL TOGETHER



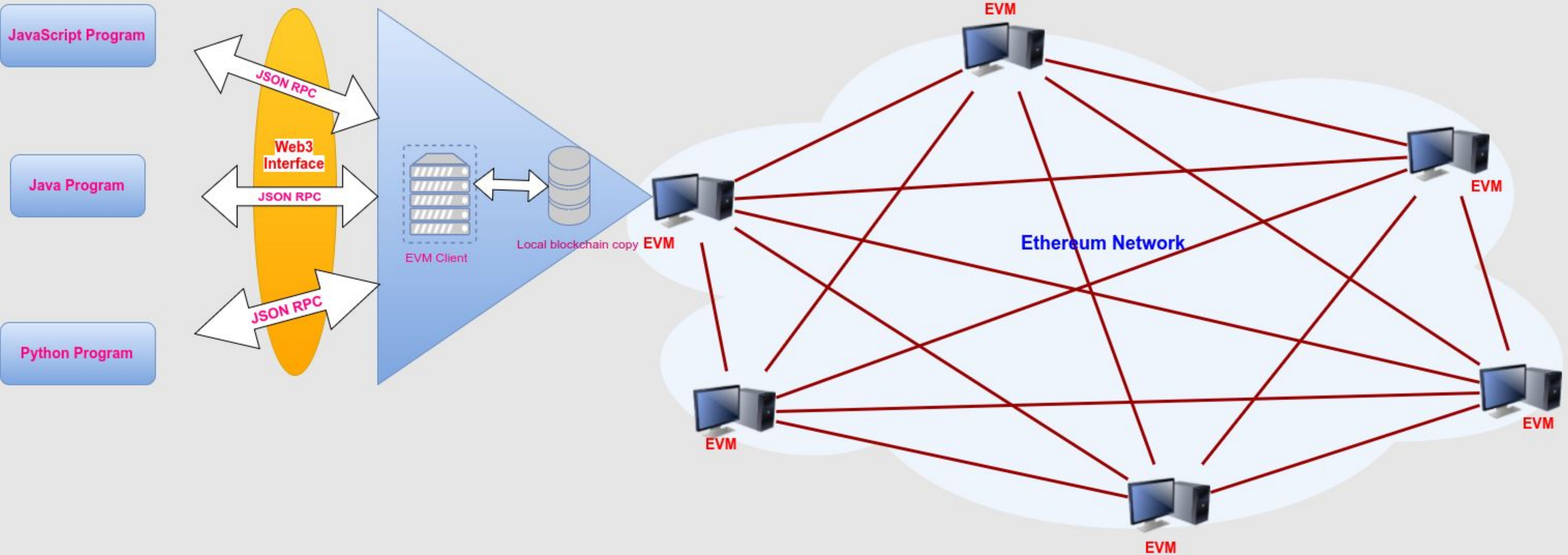


LET'S GET STARTED

READING AND WRITING TO THE CHAIN

- Ethers.js is our key to reading and writing the Ethereum blockchain
- Uses JSON RPC to talk to chain (Remote Procedural Call)
- Need to connect to an Ethereum node - can use:
 - Infura
 - Alchemy
 - Geth

Ethereum



AUTHOR: MATT PENG
UPDATED: DANIEL GUSHCHYAN

*All of the following code can be done in your terminal or a javascript file in your text editor



FIRING UP ETHERS.JS

USING METAMASK

- Let's start using ethers.js
- The easiest way to start using ethers is with MetaMask, which provides a Provider and Signer.

```
// A Web3Provider wraps a standard Web3 provider, which is
// what MetaMask injects as window.ethereum into each page
const provider = new ethers.providers.Web3Provider(window.ethereum)

// MetaMask requires requesting permission to connect users accounts
await provider.send("eth_requestAccounts", []);

// The MetaMask plugin also allows signing transactions to
// send ether and pay to change state within the blockchain.
// For this, you need the account signer...
const signer = provider.getSigner()
```



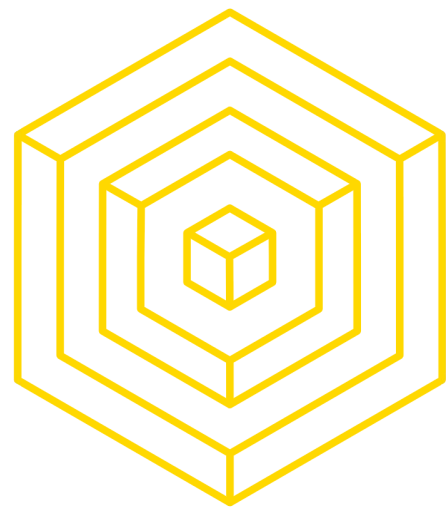
FIRING UP ETHERS.JS

USING APIs

- Another common method is to use a JSON RPC API provider.
 - Examples include Infura, Etherscan, Alchemy

```
const provider = new ethers.providers.WebSocketProvider(  
  `wss://eth-mainnet.g.alchemy.com/v2/${process.env.ALCHEMY_KEY}`  
);
```

```
// Connect to mainnet (homestead)  
provider = new AlchemyProvider();  
  
// Connect to the goerli testnet  
// (see EtherscanProvider above for other network examples)  
provider = new AlchemyProvider("goerli");  
  
// Connect to mainnet with an API key (these are equivalent)  
provider = new AlchemyProvider(null, apiKey);  
provider = new AlchemyProvider("homestead", apiKey);
```

SIMPLE ETHERS.JS CALL ACCOUNTS

- Having a Provider grants a **read-only** connection to the blockchain. Let's query some data!
 - For example, we can get the balance of an account with ethers.js:

```
await provider.getBalance("dangush.eth");  
// { BigNumber: "36426320346873870455" }
```




READ DATA FROM SMART CONTRACTS

GRABBING CONTRACTS

The **Contract** class is an abstraction of code that has been deployed to the blockchain.

A **Contract** may be sent transactions, which will trigger its code to be run with the input of the transaction data.

- Connecting to a contract requires an address, ABI, and *signer or provider*
 - By passing in a Provider, the returned object will have read-only access to the contract.
 - By passing in a Signer, we will be able to send transactions to write to it as well.

```
const ABI = require("./abi.json");

const usdcAddress = "0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48"; ///USDC Contract

const provider = new ethers.providers.WebSocketProvider(
  `wss://eth-mainnet.g.alchemy.com/v2/${process.env.ALCHEMY_KEY}`
);

const contract = new ethers.Contract(usdcAddress, ABI, provider);
```



CONT...

GRABBING CONTRACT FEATURES

- Now that a variable has been assigned to the contract using ethers.js, we can call public functions that the smart contract has within it
- Since our JavaScript **contract** object was instantiated based on the ABI we provided, we can now access the smart contract methods by calling the **contract** object.

```
const contract = new ethers.Contract(usdcAddress, ABI, provider);

const main = async () => {
  const name = await contract.name()
  const name = await contract.symbol()
}
```





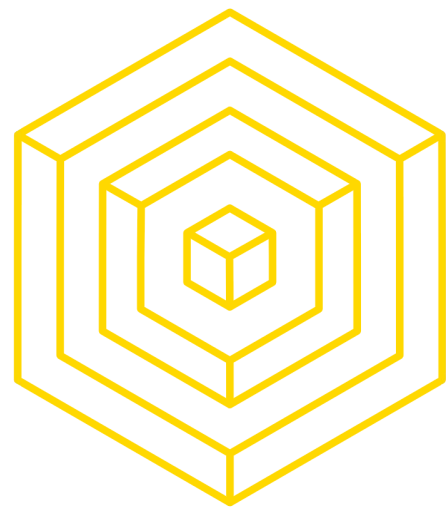
WRITING TO THE SMART CONTRACT

SENDING TRANSACTIONS

- Of course, we also want the ability to write to the contract.
- Using a contract instantiated with a **signer**, we can call write methods much like before.
- After calling the function, we receive a **txResponse** object. This represents our unconfirmed transaction.
- Calling **wait()** will wait until our transaction has been processed on Ethereum.

```
const signer = provider.getSigner();
const contract = new ethers.Contract(usdcAddress, ABI, signer);

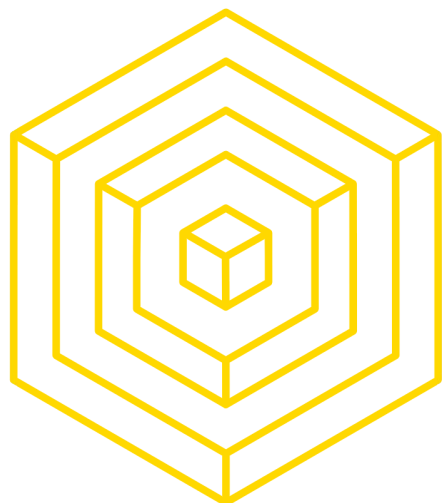
const main = async () => {
  const txResponse = await contract.setName("New Name");
  const txReceipt = await txResponse.wait;
}
```

ethers.js SUMMARY

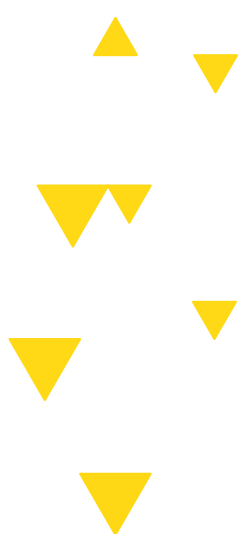
SO MUCH TO LEARN...

- So far, this has only given a glimpse of what ethers.js can do, and the tools that exist for web3 developers.
- Ethers.js also allows us to write powerful scripts for deploying contracts, interacting with Events, seeing Ethereum data, and so on.



4

DEMO?





VITAMIN/ATTENDANCE

Code: WAGMI!

